

P0019r2 : Atomic View

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0019r2
Date: 2016-03-14
Reply-to: hcedwar@sandia.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Hans Boehm
Contact: hboehm@google.com
Author: Olivier Giroux
Contact: ogiroux@nvidia.com
Author: James Reus
Contact: reus1@llnl.gov
Audience: SG1 Concurrency, Library Evolution
URL: <https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0019.rst>

1 Introduction

This paper proposes an extension to the atomic operations library [atomics] for atomic operations applied to non-atomic objects. The proposal is in five parts: (1) the concept of an atomic view, (2) application of this concept applied to single objects, (3) application of this concept applied to members of a very large array, and (4) motivating use cases and illustrative examples.

```
namespace std {  
namespace experimental {  
    template< class T > atomic_view ;  
    template< class T > atomic_array_view ;  
}}
```

Note: Feedback from SG1 Library Evolution Working Group (LEWG) on P0009r0, Polymorphic Multidimensional Array View, noted that the term *view* has the connotation of read-only. In response the P0009r0 *array_view* name has been revised to **array_ref** in P0009r1. The proposed names **atomic_view** and **atomic_array_view** may have the same feedback from LEWG, potentially resulting in a similar naming revision.

2 Atomic View Concept

A class conforming to the atomic view concept provides atomic operations for a referenced non-atomic object. These operations replicate the operations on **std::atomic<T>** [29.5 Atomic Types and 29.6 Operations on atomic types]; however, operations are const with respect to the atomic view object in contrast to the non-const and volatile operations of a **std::atomic<T>** object because an atomic view does not own (neither exclusively nor shared) the referenced non-atomic object.

The following *atomic-view-concept* specification is included to define requirements for types conforming to the *atomic-view-concept* and does not imply the existence of a template class of this name. This specification is to appear in 29.5 Atomic Types.

```
template< class T >  
struct atomic-view-concept {  
    // The following members satisfy the specifications in 29.6 Operations on atomic types,  
    // with the difference that member functions are non-volatile and const.  
    bool is_lock_free() const noexcept;  
    void store( T , memory_order = memory_order_seq_cst ) const noexcept;  
    T load( memory_order = memory_order_seq_cst ) const noexcept;  
    operator T() const noexcept ;  
    T exchange( T , memory_order = memory_order_seq_cst ) const noexcept;  
    bool compare_exchange_weak( T& , T , memory_order , memory_order ) const noexcept;  
    bool compare_exchange_strong( T& , T , memory_order , memory_order ) const noexcept;  
    bool compare_exchange_weak( T& , T , memory_order = memory_order_seq_cst ) const noexcept;  
    bool compare_exchange_strong( T& , T , memory_order = memory_order_seq_cst ) const noexcept;
```

T operator=(T) const noexcept ;

// The following member corresponds to specification of P0152R0.
static constexpr bool is_always_lock_free = *implementation defined* ;

// Alignment requirement for objects of type T
static constexpr size_t minimum_required_alignment = *implementation defined* ;

// The following member operator does not appear in atomic<T>
// it is similar to the std::unique_ptr operator.
explicit constexpr operator bool () const noexcept;

};

```
template<> struct atomic-view-concept < integral > {  
    // The following members satisfy the specifications in 29.6 Operations on atomic types,  
    // with the difference that member functions are non-volatile and const.  
  
    integral fetch_add( integral , memory_order = memory_order_seq_cst) const noexcept;  
    integral fetch_sub( integral , memory_order = memory_order_seq_cst) const noexcept;  
    integral fetch_and( integral , memory_order = memory_order_seq_cst) const noexcept;  
    integral fetch_or( integral , memory_order = memory_order_seq_cst) const noexcept;  
    integral fetch_xor( integral , memory_order = memory_order_seq_cst) const noexcept;  
  
    integral operator++(int) const noexcept;  
    integral operator--(int) const noexcept;  
    integral operator++() const noexcept;  
    integral operator--() const noexcept;  
    integral operator+=( integral ) const noexcept;  
    integral operator-=( integral ) const noexcept;  
    integral operator&=( integral ) const noexcept;  
    integral operator|=( integral ) const noexcept;  
    integral operator^=( integral ) const noexcept;  
};
```

Requires: Type *T* is trivially copyable.

Lock-free *atomic-view-concept* conform to the address-free property as in 29.4p3.

Constructors and assignment operators of non-lock-free *atomic-view-concept* may acquire shared resources such as concurrent locks to support atomic operations on the non-atomic object.

static constexpr size_t minimum_required_alignment

Requires: An object referenced by an *atomic-view-concept* shall be aligned to **minimum_required_alignment**. [Note: For example, an architecture may be able to support lock-free *atomic-view-concept* operations on **std::complex<double>** only if aligned to 16 bytes. - end note]

explicit constexpr operator bool () const noexcept ;

Returns: true if the *atomic-view-concept* object wraps a non-null pointer. A default constructed *atomic-view-concept* object returns false.

3 Atomic View for a Single Object

An **atomic_view<T>** object is used to perform atomic operations on the viewed non-atomic object. The intent is for **atomic_view<T>** to provide the best-performing implementation of *atomic-view-concept* operations for the type **T**. All atomic operations on an instance of **atomic_view<T>** are atomic with respect to any other instance that views the same object, as defined by equality of pointers to that object.

Introducing concurrency within legacy codes may require replacing operations on existing non-atomic objects with atomic operations such that the non-atomic object cannot be replaced with a *std::atomic* object.

An object may be heavily used non-atomically in well-defined phases of an application. Forcing such objects to be exclusively

std::atomic would incur an unnecessary performance penalty.

This specification is to appear in a new section 29.# Atomic Views.

```
template< class T > struct atomic_view { // conforms to atomic view concept

    explicit atomic_view( T & obj ); // wrapping constructor is NOT noexcept

    constexpr atomic_view() noexcept ;
    atomic_view( atomic_view && ) noexcept ;
    atomic_view( const atomic_view & ) noexcept ;
    atomic_view & operator = ( atomic_view && ) noexcept ;
    atomic_view & operator = ( const atomic_view & ) noexcept ;
    ~atomic_view();
};
```

Requires: Type *T* is trivially copyable.

[Note: The intent is for *atomic-view-concept* operations to directly update the referenced object. The *wrapping constructor* may acquire a resource, such as a lock from a collection of address-sharded locks, to perform atomic operations. Such **atomic_view** objects are not lock-free and not address-free. When such a resource is necessary subsequent copy and move constructors and assignment operators may reduce overhead by copying or moving the previously acquired resource as opposed to re-acquiring that resource. – end note]

constexpr atomic_view<T>::atomic_view() noexcept;

Effects: This instance does not reference an object and therefore **operator bool() == false**.

atomic_view<T>::atomic_view(T & obj);

Requires: The referenced non-atomic object **obj** shall be aligned to **minimum_required_alignment**. The lifetime (3.8) of an **atomic_view<T>** instance shall not exceed the lifetime of the referenced non-atomic object. Multiple instances of an *atomic_view* may be constructed referencing the same object. All accesses of an **atomic_view** referenced object shall occur thru an **atomic_view** as long an **atomic_view** exists that references that object. If the **atomic_view** wrapped object is of a class or aggregate type then members of that object shall not be wrapped by an **atomic_view** object. If he **atomic_view** wrapped object is a member of an array that array shall not be wrapped by an **atomic_array_view**.

Effects: References the non-atomic object. Atomic operations on this instance are atomic with respect to atomic operations on any **atomic_view** instance that references the same object. May acquire shared resources such as a lock associated with the referenced object.

Throws: If *atomic-view-concept* operations cannot be supported for the referenced object. [Note: For example, if the referenced object is not properly aligned or has automatic storage duration within an accelerator coprocessor (e.g., a GPGPU) execution context. - end note] If resource acquisition, such as a lock, is required and fails.

atomic_view<T>::atomic_view(atomic_view && rhs) noexcept ;
atomic_view<T>::atomic_view(const atomic_view & rhs) noexcept ;
atomic_view<T> & atomic_view<T>::operator = (atomic_view && rhs) noexcept ;
atomic_view<T> & atomic_view<T>::operator = (const atomic_view & rhs) noexcept ;

Effects: If *rhs* references an object then this instance references the same object otherwise this instance does not reference an object.

atomic_view<T>::~~atomic_view() noexcept ;

Effects: Releases shared resources that may have been acquired.

4 Atomic View for a Very Large Array

High performance computing (HPC) applications use very large arrays. Computations with these arrays typically have distinct phases that allocate and initialize members of the array, update members of the array, and read members of the array. Parallel algorithms for initialization (e.g., zero fill) have non-conflicting access when assigning member values. Parallel algorithms for updates have conflicting access to members which must be guarded by atomic operations. Parallel algorithms with read-only access require best-performing streaming read access, random read access, vectorization, or other guaranteed non-conflicting

HPC pattern.

An *atomic_array_view* object is used to perform atomic operations on the viewed non-atomic members of the array. The intent is for *atomic_array_view* to provide the best-performing implementation of atomic-view-concept operations for the members of the array.

Recall that any number of *atomic_view* entities may independently *wrap construct* the same underlying object and all *atomic-view-concept* operations performed thru any of those **atomic_view** entities are atomic for the referenced object. In contrast, only one **atomic_array_view** entity may **wrap construct** an array and thus *atomic-view-concept* operations must be performed thru that entity or **atomic_array_view** entities transitively copy constructed, move constructed, copy assigned, or move assigned from that originating *wrap constructed* **atomic_array_view** entity. This allows a non-lock-free *atomic_array_view* to acquire resources, such as a set of locks, that are exclusively associated with the wrapped array. When such a resource is necessary subsequent copy and move constructors and assignment operators may reduce overhead by copying or moving the previously acquired resource as opposed to re-acquiring that resource. The intent is to enable reduction of the time and space overhead associated with of managing such non-lock-free resources.

Note that an **atomic_array_view** is similar to **string_view** (N4480 Section 7) in that it wraps or references a contiguous set of objects; however, the reference is non-constant.

This specification is to appear in a new section 29.# Atomic Views.

```
template< class T > struct atomic_array_view {

    // Alignment requirement for objects of type T
    static constexpr size_t alignment = implementation defined ;

    static constexpr bool is_always_lock_free = implementation defined ;
    bool is_lock_free() const noexcept ;

    // Returns true if the view wraps an array and member access is valid.
    explicit constexpr operator bool() const noexcept ;

    atomic_array_view( T * , size_t ); // Wrapping constructor is NOT noexcept
    constexpr atomic_array_view() noexcept ;
    atomic_array_view( atomic_array_view && ) noexcept ;
    atomic_array_view( const atomic_array_view & ) noexcept ;
    atomic_array_view & operator = ( atomic_array_view && ) noexcept ;
    atomic_array_view & operator = ( const atomic_array_view & ) noexcept ;
    ~atomic_array_view();

    size_t size() const noexcept ;

    using reference = implementation-defined-atomic-view-concept-type ;

    reference operator[] ( size_t ) const noexcept ;
};
```

Requires: Type *T* is trivially copyable.

using reference = *implementation-defined-atomic-view-concept-type* ;

Requires: The **reference** type conforms to *atomic-view-concept* for type *T*. [Note: The **reference** type is *not* required to be *atomic_view<T>*. - end note]

static constexpr bool is_always_lock_free = *implementation defined* ;
bool atomic_array_view<T>::is_lock_free() const noexcept ;

Returns: Whether atomic operations on members are (always) lock free.

constexpr atomic_array_view<T>::atomic_array_view() noexcept;

Effects: The constructed **atomic_array_view** does not reference an array and therefore **size()** == 0.

atomic_array_view<T>::atomic_array_view(T * ptr , size_t N);

Requires: If 0 < N the array referenced by [ptr .. ptr+N) shall be within a contiguously allocated set of objects (8.3.4p1)

and shall be aligned to **minimum_required_alignment**. This *wrapping constructor* shall not be applied to any subset of the array, including the entire array, as long as an **atomic_array_view** entity exists wrapping that array. An **atomic_view** shall not exist for any member of the array as long as an **atomic_array_view** entity exists for that array. All accesses of the array's members shall occur through an **atomic_array_view<T>::reference** as long as an **atomic_array_view** exists for that array.

Effects: If $0 < N$ the *wrapping constructor* wraps the referenced contiguously allocated array [ptr .. ptr+N); otherwise the **atomic_array_view** does not reference an array. Atomic operations on members of this instance are atomic with respect to atomic operations on members any **atomic_array_view** instance that references the same array. May acquire shared resources such as a set of locks.

Throws: If *atomic-view-concept* operations cannot be supported for members of the referenced array. [Note: For example, if the referenced array is not properly aligned or has automatic storage duration within an accelerator coprocessor (e.g., a GPGPU) execution context. - end note] If resource acquisition, such as a set of locks, is required and fails.

```
atomic_array_view<T>::atomic_array_view( atomic_array_view && rhs ) noexcept ;
atomic_array_view<T>::atomic_array_view( const atomic_array_view & rhs ) noexcept ;
atomic_array_view<T> & atomic_array_view<T>::operator = ( atomic_array_view && rhs ) noexcept ;
atomic_array_view<T> & atomic_array_view<T>::operator = ( const atomic_array_view & rhs ) noexcept ;

atomic_array_view<T>::~atomic_array_view() noexcept ;
```

Effects: Releases shared resource that may have been acquired.

```
atomic_array_view<T>::reference atomic_array_view<T>::operator[]( size_t i ) const noexcept ;
```

Requires: $i < \text{size}()$ and the lifetime of the returned **reference** object, copied **reference** object, or moved **reference** object shall not exceed the lifetime of the associated **atomic_array_view**. [Note: Analogous to the lifetime of an iterator with respect to the lifetime of the associated container. - end note]

Returns: An instance of **reference** type that references the i-th member of the referenced array, where indexing is zero-based. [Note: The intent is for efficient generation of the returned *atomic-view-concept* object with respect to resources required to support non-lock-free *atomic-view-concept* operations. – end note]

5 Notes and Examples

Under the HPC use case the member access operator, proxy type constructor, or proxy type destructor will be frequently invoked; therefore, an implementation should trade off decreased overhead in these operations versus increased overhead in the wrapper constructor and final destructor.

Usage Scenario for **atomic_array_view<T>**

- A very large array of trivially copyable members is allocated.
- A parallel algorithm initializes members through non-conflicting assignments.
- The array is wrapped by an **atomic_array_view<T>**.
- One or more parallel algorithms update members of the array through atomic view operations.
- The **atomic_array_view<T>** is destructed.
- Parallel algorithms access array members through non-conflicting reads, writes, or updates.

Example:

```
// atomic array view wrapper constructor:
atomic_array_view<T> array( ptr , N );

// atomic operation on a member:
array[i].atomic-operation(...);

// atomic operations through a temporary value
// within a concurrent function:
atomic_array_view<T>::reference x = array[i];
x.atomic-operation-a(...);
x.atomic-operation-b(...);
```

Possible interface for **atomic_array_view<T>::reference**

```
struct implementation-defined-proxy-type { // conforms to atomic view concept
```

```
// Construction limited to move
implementation-defined-proxy-type(implementation-defined-proxy-type && ) = noexcept ;
~implementation-defined-proxy-type();

implementation-defined-proxy-type() = delete ;
implementation-defined-proxy-type( const implementation-defined-proxy-type & ) = delete ;
implementation-defined-proxy-type &
    operator = ( const implementation-defined-proxy-type & ) = delete ;
};
```

Wrapping constructor options for **atomic_array_view**<T>

A wrapping constructor of the form (T*begin, T*end) could be valid. However, the (T*ptr, size_t N) version is preferred to minimize potential confusion with construction from non-contiguous iterators. Wrapping constructors for standard contiguous containers would also be valid. However, such constructors could have potential confusion as to whether the **atomic_array_view** would or would not track resizing operations applied to the input container.